Assignment: Payment status inbox

Build a small app where an operator starts a card payment and follows its status. A fake provider accepts the payment immediately and reports the outcome later by callback.

Deliver: running backend, running Vue page, backend tests, README with start/test commands, one-page NOTES.md.

Out of scope: login, Kafka, Kubernetes, a real provider, component libraries, visual design.

Tech

- Java 21, Spring Boot 3.2+, Maven/Gradle wrapper committed.
- Vue 3, TypeScript, lockfile committed.
- Runs fully locally and offline: H2/SQLite or docker compose with Postgres, no external calls, no secrets. Backend on `localhost:8080`, frontend on `localhost:3000`.

Backend

Fake provider (class in the backend): returns a `providerPaymentId` and sends its own callback after 2-5 s. The outcome is configurable (`CAPTURED`, `DECLINED`, `ERROR`, or an unknown value). Document how in the README.

`POST /payments`: `amount`, `currency`, `customerReference`. Validate (`400`). Creates a `PENDING` payment, calls the provider, stores `providerPaymentId`, returns `201`. No `double` / `float` for money.

`POST /provider-callbacks`:

```
{ "eventId": "evt-1", "providerPaymentId": "prov-123", "status": "CAPTURED", "occurredAt": "20
```

Provider status	Our status
AUTHORIZED	PENDING
CAPTURED	SUCCEEDED
DECLINED	FAILED
ERROR, anything else (incl. null)	UNKNOWN

Returns `200` when accepted (even if nothing changes), `404` for unknown `providerPaymentId`, `400` for invalid payload.

`GET /payments/{id}`: `id`, `status`, `providerStatus` (latest), `providerPaymentId`, `lastCallbackOccurredAt` (last applied callback, `null` if none), `amount`, `currency`, `customerReference`. `404` if unknown.

Rules

1. The same `eventId` is applied only once.
2. A callback with `occurredAt` earlier than or equal to the last applied one is ignored.
3. `UNKNOWN` must not replace `SUCCEEDED` or `FAILED`. A later `CAPTURED` or `DECLINED` can replace the previous known outcome.
4. A callback for an unknown `providerPaymentId` is rejected.

Decision for you: does a callback suppressed by rule 3 still update `lastCallbackOccurredAt`, `providerStatus` and the recorded `eventId`? Decide, test, and explain in NOTES.md.

Frontend

One page: a form (amount, currency, customer reference) and a status panel.

- Show clearly differently: submitting, pending, succeeded, failed, unknown, backend unreachable.
- Poll `GET /payments/{id}` every ~2 s while `PENDING` or `UNKNOWN`. Stop on `SUCCEEDED` or `FAILED`.
- Clean up polling on unmounting and when a new payment is created.

Tests

Backend tests for rules 1-3 (including equal timestamps).

NOTES.md (one page)

1. Three decisions you made yourself (include the one above if not among them)
2. One thing an AI tool got wrong, and what you changed (be specific)
3. What you would not ship in a production payment service

Submit

Git repo or zip. The README gives Java/Node versions, start and test commands, and how to choose the provider outcome.